INTRODUCCIÓN

Carlos Pérez

Departamento de Lenguajes y Sistemas Informáticos
Universidad de Alicante
Curso 2026-2027



Esta obra se publica con una licencia BY-NC-SA Creative Commons License

Index

1. Estructura de la asignatura
2. Introducción a la estructura de datos
3. Colecciones de datos en Java y C++
4. Algoritmia
5. Bibliografía

Index

1. Estructura de la asignatura
2. Introducción a la estructura de datos
3. Colecciones de datos en Java y C++
4. Algoritmia
5. Bibliografía

Introducción

- **Estructura de Datos y Algoritmia (EDA)**: asignatura obligatoria
- **Campus Virtual (UACloud)**
  - Anuncios: aclaraciones o noticias sobre cualquier tema relacionado con la asignatura
  - Tutorías: tutorías virtuales o reserva de cita presencial
  - Evaluación: publicación de notas de exámenes y prácticas
- **Moodle UA**
  - Materiales docentes: presentaciones, enunciados de prácticas, ejercicios, etc
- **Web del DLSI**
  - Entrega de prácticas (`http://pracdlsi.dlsi.ua.es/`)

Profesorado

- **Carlos Pérez Sancho (teoría y prácticas)**
- **Francisco Moreno Seco (prácticas)**
- **Pablo Cubillas Alarcón (prácticas)**
- **Miquel Esplà Gomis (prácticas)**
- **Leopoldo Pla Sempere (prácticas)**

Departamento de Lenguajes y Sistemas Informáticos

lsi Department of Software and Computing Systems

Objetivos de la asignatura

- Conocer, implementar y gestionar **estructuras de datos**, y seleccionar las más adecuadas para cada problema.
- Determinar la **eficiencia de algoritmos** y usar de forma eficiente los distintos métodos de **ordenación y búsqueda**.
- Conocer y saber emplear técnicas fundamentales de **diseño de algoritmos** que permitan comprender la naturaleza de los problemas independientemente de la implementación.

Temario

- Introducción a la estructura de datos y la algoritmia
- Análisis de complejidad de los datos
- Estructuras de datos lineales
- Estructuras de datos no lineales
- Algoritmos de ordenación
- Estrategias algorítmicas

Prácticas

- **Prácticas individuales**
- **3 prácticas**
  - en C++: 1 y 2
  - en Java: 3

Evaluación en enero

- **Teoría:**
  - Examen final con nota mínima de 4 sobre 10 (50% nota total)
- **Prácticas:** 4 prácticas (50% nota total). Sin nota mínima en cada práctica individualmente.
  - Práctica 1: 20% de la nota de prácticas
  - Práctica 2: 50% de la nota de prácticas
  - Práctica 3: 30% de la nota de prácticas

  Calculada esta nota ponderada, su valor tiene que ser un 4 como mínimo para promediar con la nota de teoría.

Prácticas

- **Desarrollo de la práctica**
  - Se entregará un enunciado de la práctica con un plazo de varias semanas para su desarrollo.
  - La práctica se entregará en la fecha indicada en el enunciado, a través de la web del DLSI.
  - Se proporcionará un **conjunto de pruebas básico** para comprobar el correcto funcionamiento de la práctica.
  - La práctica entregada **no se evaluará directamente**.
- **Examen presencial de la práctica**
  - Consistirá en una modificación o ampliación de la práctica entregada.
  - La evaluación se hará con un **conjunto de pruebas más amplio** que el proporcionado en la entrega, evaluando tanto el enunciado original como la modificación o ampliación.
- **Asistencia obligatoria** a prácticas (máximo 3 faltas sin justificar)
- **En caso de detectarse indicios de copia[1], se suspenderá la práctica correspondiente a todas las personas implicadas.**

[1] Codigo similar desarrollado por más de una persona o por academias

Evaluación en julio (convocatoria extraordinaria)

- **Teoría:**
  - Examen final con nota mínima de 4 sobre 10 (50% nota total)
- **Prácticas:**
  - Examen final con nota mínima de 4 sobre 10 (50% nota total)

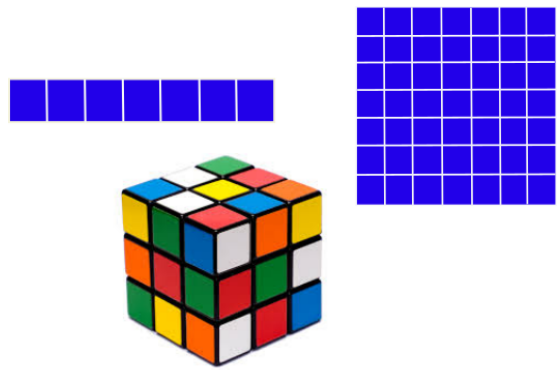
Index

1. Estructura de la asignatura
2. Introducción a la estructura de datos
3. Colecciones de datos en Java y C++
4. Algoritmia
5. Bibliografía

Organización de una colección de objetos

Hasta ahora habéis trabajado con un par de estructuras de datos sencillas:

- tipos simples: enteros, reales, boolean ...
- cadenas de caracteres
- arrays de objetos (uni|bi|multi)-dimensionales



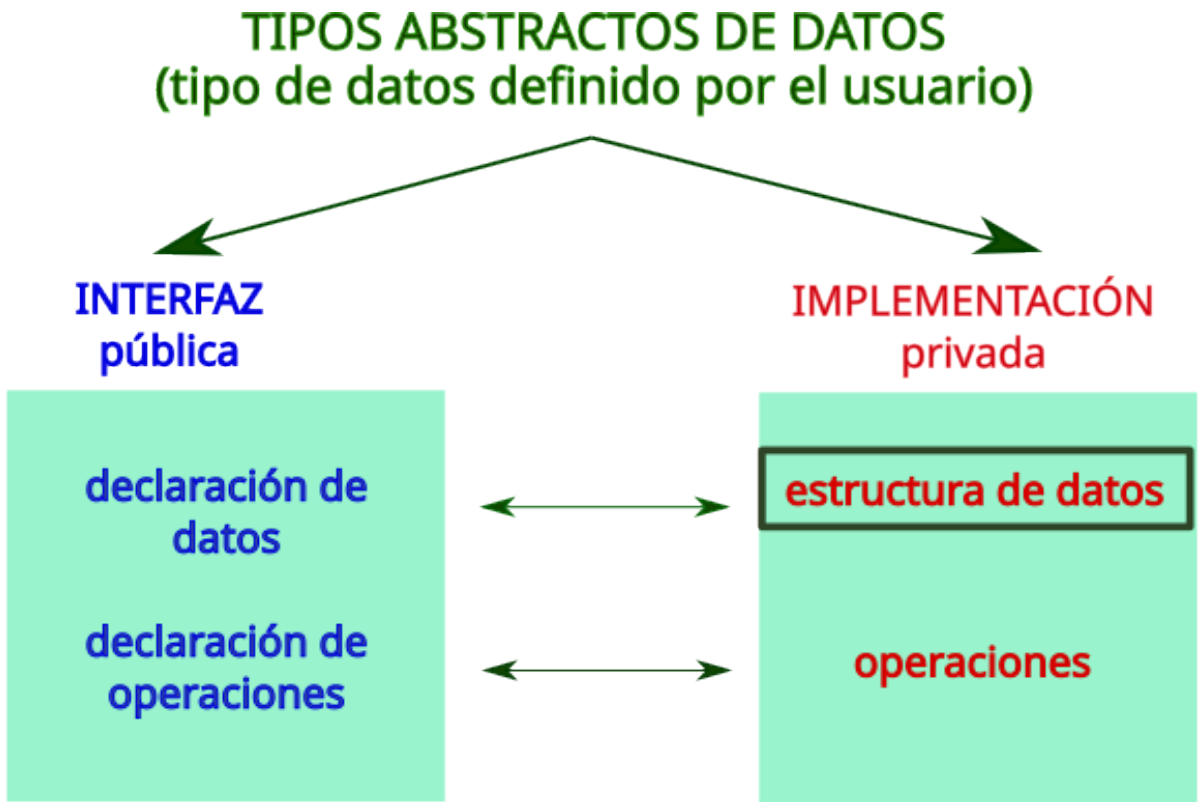
Suficiente en principio, pues podemos almacenar cosas como

- la lista de alumnos de un curso
- la colección de coordenadas de un mapa
- todas las palabras que forman un diccionario, etc

Problema: responder a preguntas sobre las colecciones de objetos almacenados en las anteriores estructuras nos llevará a **respuestas lentísimas** en muchos casos.

¿Qué son las estructuras de datos?

Organización de datos con diferentes formas de acceso.



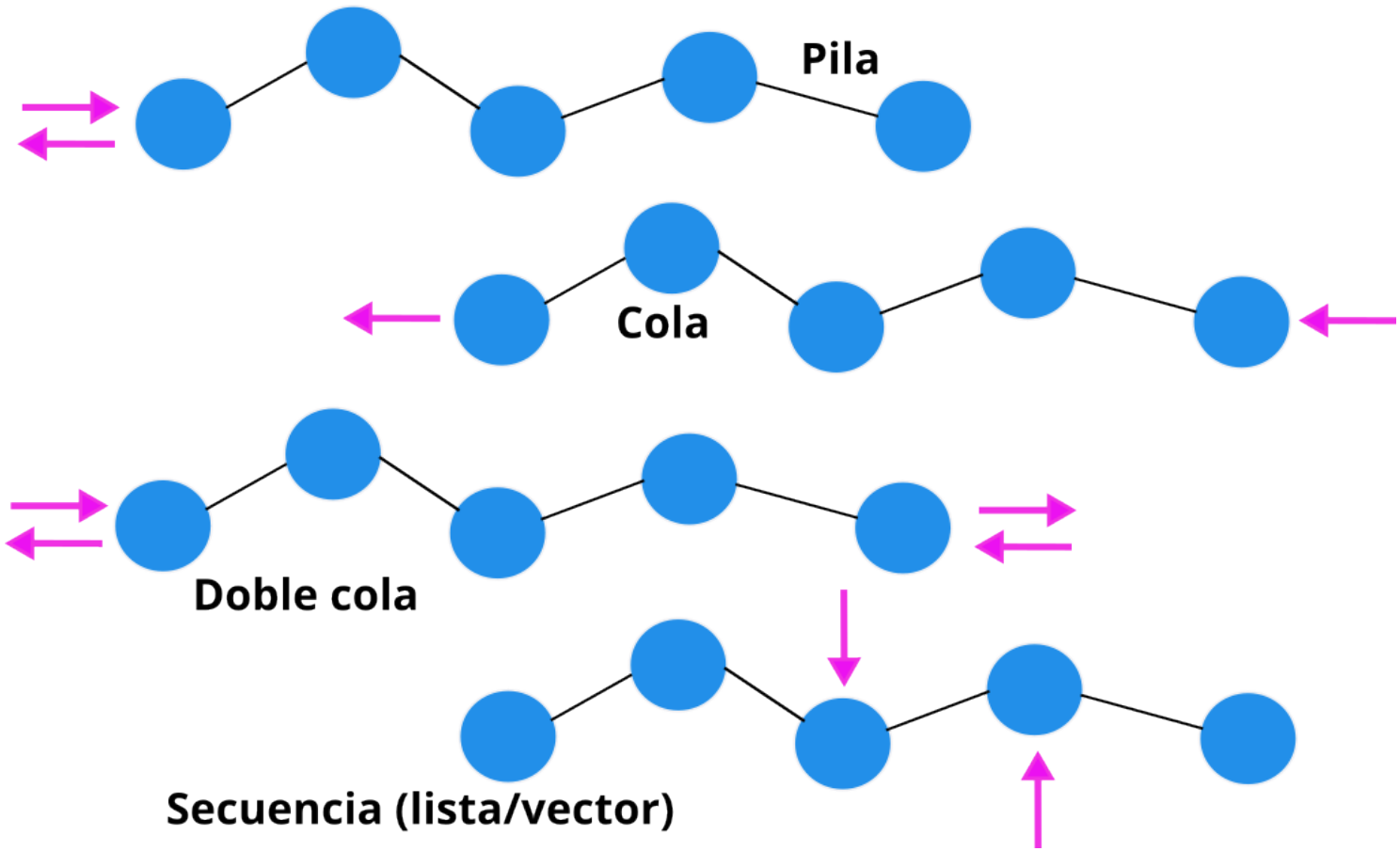
- definición de interfaz (encapsulación, modularidad, reutilización)
- varias implementaciones posibles (adaptación a requisitos)

Propiedades

- Los **programas** son **independientes de la forma** en que se implementan los tipos de datos construidos por el usuario.
- Un **criterio importante** para elegir una estructura de datos es la **complejidad o tiempo de ejecución**.
- Las **operaciones básicas** que se definen en cualquier estructura de datos son: búsqueda, inserción y borrado.
- Las ESTRUCTURAS DE DATOS existentes se diferencian entre sí por las **relaciones que se establecen entre sus elementos**.

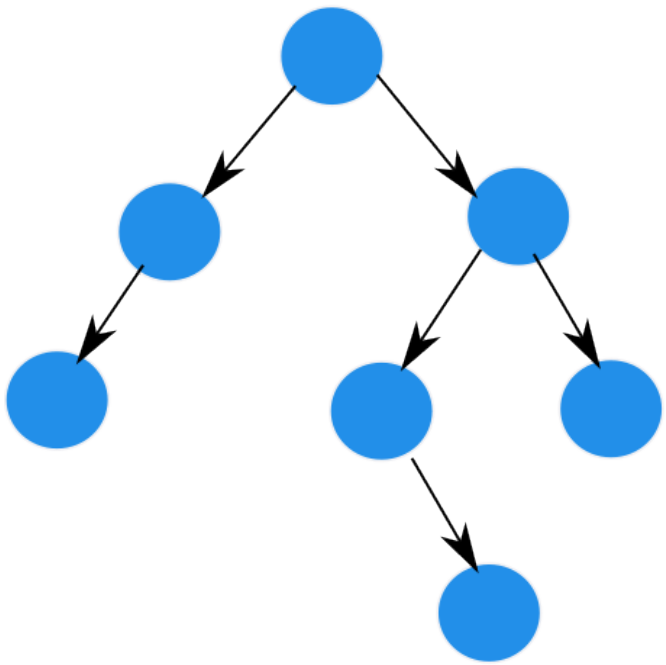
Estructuras de datos lineales

En general, lo que tenemos son secuencias, una **relación uno a uno** entre los elementos almacenados (vectores, pilas, colas, listas ...).

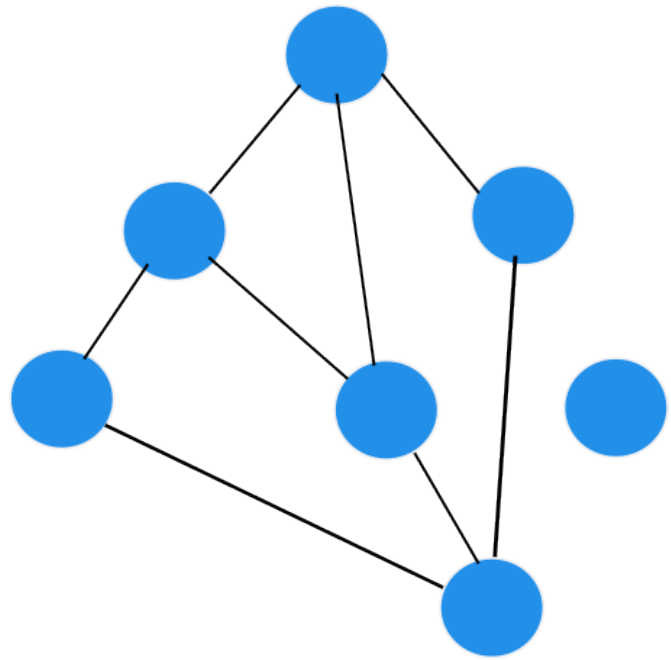


Estructuras de datos no lineales

En este caso podemos tener relaciones **uno a muchos o muchos a muchos** (árboles, colas de prioridad, grafos, mapas ...)



uno a muchos (árbol)



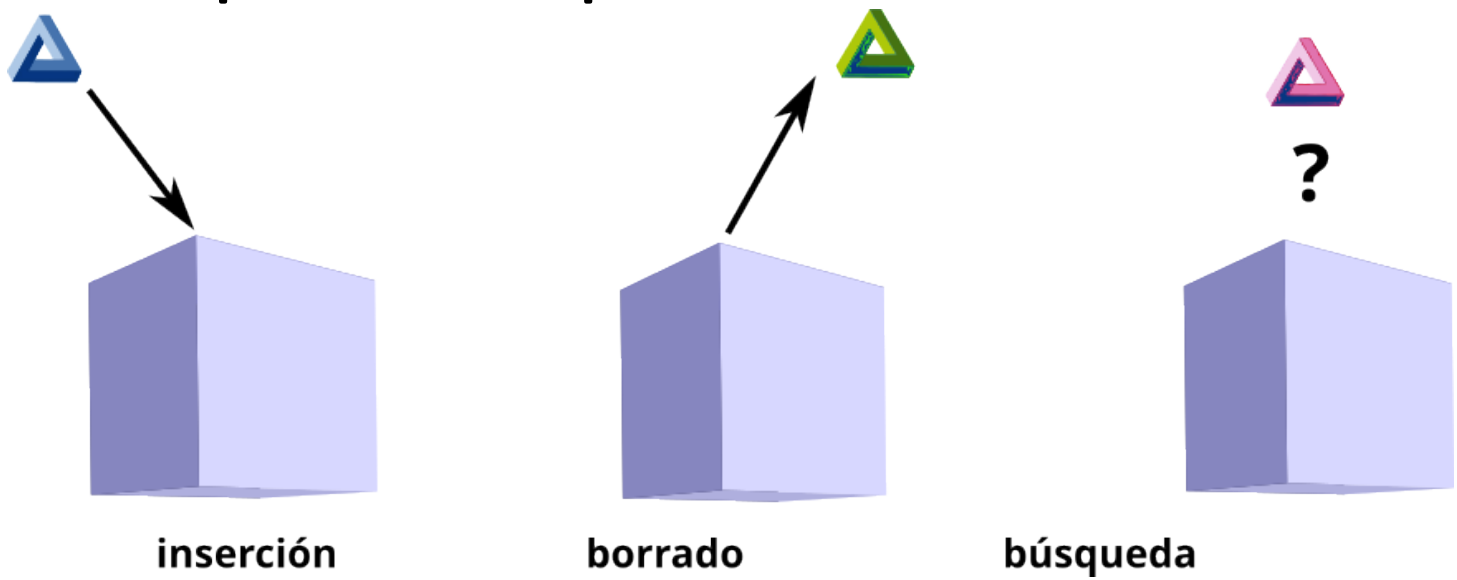
muchos a muchos (grafo)

Index

1. Estructura de la asignatura
2. Introducción a la estructura de datos
3. Colecciones de datos en Java y C++
4. Algoritmia
5. Bibliografía

Colecciones de datos

Operaciones imprescindibles para GESTIONAR colecciones de datos



Operaciones imprescindibles para ORGANIZAR colecciones de datos

- relación de igualdad (ver si dos objetos son iguales)
- relación de orden (ver si un objeto es anterior o posterior a otro)

Relación de igualdad en Java

Para poder comparar, los objetos en Java disponen del método `equals` definido en la superclase `Object`

public boolean equals(Object ob)

Si definimos nuestras propias clases, debemos redefinir este método, ya que por omisión compara las referencias a los objetos.

Ejemplo:

```
public class Pixel {
  private int x, y;
  public boolean equals (Object o){
    if (o instanceof Pixel) {
      Pixel c = (Pixel) o;
      return ( this.x == c.x && this.y == c.y );
    } else {
      return false;
    }
  }
}
```

Relación de igualdad en C++

Si definimos nuestras propias clases, debemos redefinir el operador ==, ya que por omisión se comparan las referencias a los objetos.

Ejemplo:

```
//En Pixel.h
class Pixel{
  private:
      int x, y;
  public:
      Pixel(int i, int j){ x = i; y = j;}
      bool operator==(const Pixel &);
}

//En Pixel.cc
bool Pixel::operator==(const Pixel &v){
    if(v.x == x && v.y == y) return true;
    else return false;
}

//En main.cc
    if( p==q ) cout << "p y q son iguales" << endl;
```

Relación de orden en Java

La interfaz **Comparable** especifica una operación de comparación asociada a los objetos (tipo T) de las clases que implementan esta interfaz

public int compareTo(T ob)

- al hacer una llamada del tipo x.compareTo(y) la salida es
  - 0 si x es igual a y
  - <0 si x es menor a y
  - >0 si x es mayor a y
- debe imponer una relación de orden total
- debe ser consistente con equals(), es decir

(x.compareTo(y)==0) es equivalente a (x.equals(y))

Relación de orden: ejemplo de compareTo en Java

```
public class Pixel implements Comparable<Pixel> {
  private int x, y;

  public int compareTo (Pixel o) {
    boolean result = (x == o.x);
    if (result) {
      return (y<o.y?-1 : (y==o.y?0:1));
    }
    else return (x < o.x ? -1 : 1);
  }
}

//en main
Pixel p,q;
p = new Pixel(1,2); q = new Pixel(1,3);
if( p.compareTo(q) < 0 )
    System.out.println(``p es menor que q'');
```

Relación de orden en C++

Redefinición de operadores: $==$, $!=$, $<$, $>$, $<=$, $>=$

consideramos que $(x, y) < (z, t)$ si $x < z$, o si siendo $x = z$ entonces $y < t$

```
//En Pixel.cc
bool Pixel::operator<(const Pixel &v){
    bool dev = false;
    if(x < v.x) dev = true;
      else if(x==v.x && y < v.y ) dev = true;
    return dev;
}

//En main.cc
    if(p<q) cout << "p es menor que q" << endl;
```

Tipos genéricos o parametrizados

- Permiten definir un tipo sin especificar todos los tipos concretos que éste puede usar o gestionar.
- Se utilizan especialmente para implementar **tipos abstractos de datos**: pilas, colas, vectores, listas, mapas ... permitiendo almacenar distintos tipos de elementos según sean instanciados en tiempo de compilación.

Bibliotecas de plantillas en Java y C++:

- **Java: Java Collection Framework (en el paquete java.util)**

```
ArrayList<String> v1 = new ArrayList<String>();
ArrayList<Integer> v2 = new ArrayList<Integer>();
```

- **C++: Standard Template Library (STL)**

```
vector<string> v1;
vector<int> v2;
```

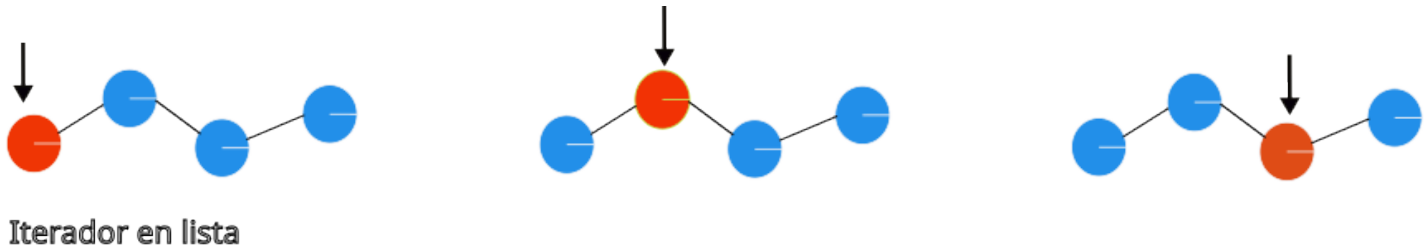
Clases contenedoras en Java y C++

tipos de datos	**C++**	**Java**	**operaciones**
arrays dinámicos	vector	ArrayList	inserción y borrados rápidos al final. Acceso directo
colas	deque	Deque*	inserciones y borrados rápidos en ambos extremos
listas enlazadas	list	LinkedList	lista de enlace doble
conjuntos	set multiset	TreeSet HashSet	búsqueda por elemento
arrays asociativos	map multimap	TreeMap HashMap	búsqueda por clave

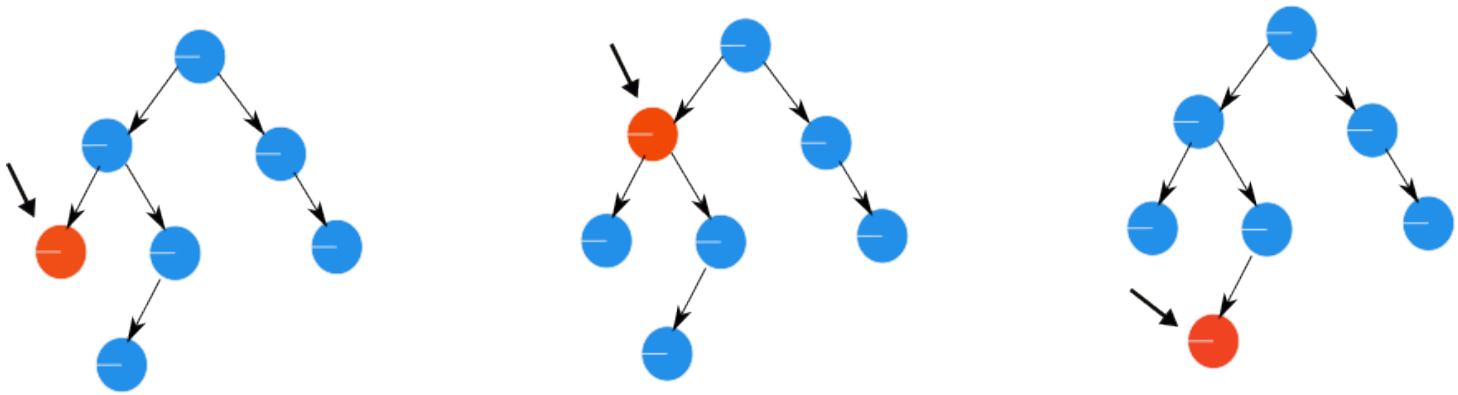
*interface

Sobre iteradores

Un **iterador** definido sobre una clase permite visitar todos sus elementos uno por uno. El iterador se puede ver como un "puntero inteligente" (o cursor) que recorre una secuencia de elementos.



Iterador en lista



Iterador en TreeSet

Index

1. Estructura de la asignatura
2. Introducción a la estructura de datos
3. Colecciones de datos en Java y C++
4. Algoritmia
5. Bibliografía

¿Qué es la algoritmia?

Tratamiento sistemático de técnicas fundamentales para el diseño y análisis de algoritmos eficientes.

Los computadores son cada vez más rápidos y más baratos:

- se resuelven problemas de cálculo antes impensables;
- inconscientemente se tiende a restar importancia al concepto de eficiencia;

Existen problemas que seguirán siendo **intratables (NP)** si se aplican ciertos algoritmos

$\rightarrow$ hay que buscar soluciones eficientes!

Ejemplo de problema intratable

Enunciado del problema del viajante. Dada una lista de ciudades y las distancias entre cada par de ellas, ¿cuál es la ruta más corta posible que visita cada ciudad exactamente una vez y regresa a la ciudad origen?

Caso práctico: si N es el número de ciudades, tenemos $N!$ rutas posibles (que optimizando dan $(N-1)!/2$ rutas posibles)...

- $N = 5 \rightarrow$ 12 rutas
- $N = 10 \rightarrow$ 181.440 rutas
- $N = 30 \rightarrow$ más de $4 \cdot 10^{30}$ rutas

Un ordenador que calcule **mil millones**[2] de rutas por segundo necesitaría más de 10^{14} años para resolverlo (el Universo se creó hace menos de 10^{11} años).

`http://es.wikipedia.org/wiki/Problema_del_viajante`

[2] Dato actualizado a 2024

Nuestro objetivo

Objetivo principal: proponer técnicas (o esquemas) de diseño de algoritmos eficientes para facilitar el desarrollo de programas.

Medios para alcanzar este objetivo:

- presentar cada esquema de forma genérica, incidiendo en los principios que conducen a él;
- ilustrar el esquema mediante ejemplos concretos de algoritmos tomados de varios dominios de aplicación;

¿A quién puede interesar la algoritmia?

A todo aquél a quien le guste

- diseñar algoritmos para resolver nuevos problemas, o
- diseñar algoritmos mejores a los triviales para resolver viejos problemas

`https://arxiv.org/pdf/2504.17033`



$O(n^2)$ **coste cuadrático** $\Longrightarrow$ $O(n)$ **coste lineal**

Index

1. Estructura de la asignatura
2. Introducción a la estructura de datos
3. Colecciones de datos en Java y C++
4. Algoritmia
5. Bibliografía

Bibliografía

Además de los materiales del Campus Virtual ...

- *Estructuras de datos y algoritmos en Java*
  Adam Drozdek
  Thomson, 2007.

- *Estructura de datos en C++*
  L. Joyanes, L. Sánchez, I. Zahonero
  Mc Graw Hill, 2007.

- *Fundamentos de algoritmia*
  G. Brassard, P. Bratley
  Prentice Hall, 1997.

Bibliografía online o en pdf

- *Estructuras de datos en Java*
  L. Joyanes, I. Zahonero
  Mc Graw Hill, 2008.

- *Programación en C++. Un enfoque práctico. Serie Schaum*
  L. Joyanes, L. Sánchez
  Mc Graw Hill, 2003